

Inside Kahuna’s Distributed Transactions: Durable Intents, Validation, and Recovery in a Partitioned Key–Value Store

Andres Gutierrez

August 2026

Abstract Kahuna is a partitioned key–value store whose transaction layer combines actor-owned in-memory state, Raft-replicated metadata, hybrid logical clocks, multiversion reads, lock-based and optimistic concurrency control, and asynchronous post-commit materialization. This paper reconstructs the transaction protocol in Kahuna 1.5.3 and explains the guarantees that emerge from the interaction of its subsystems. The central persistent protocol stores a canonical transaction record on an anchor partition and a prepared intent beside every modified key. Preparation records the exact value, commit timestamp, participant manifest, expiration, and—for read-before-write operations—the validated base revision. A transaction commits only after every prepare is durable and after conflict and read-set validation. Readers consult the canonical decision while committed intents await materialization; recovery can therefore complete an acknowledged transaction without the original coordinator. The design includes an atomic single-partition fast path, heterogeneous Raft write coalescing, adaptive decision deadlines, idempotent operation admission, snapshot-retention leases, and barriers for range reconfiguration. We analyze which anomalies these mechanisms prevent, where guarantees depend on transaction options, and where they deliberately stop. In particular, optimistic predicate scans do not constitute full phantom protection, mixed persistent/ephemeral transactions are not crash-atomic, and current replica-side stale-base vetoing remains under fault-soak validation. The result is best understood not as one universal isolation level, but as a family of execution policies built over a durable-intent commit core.

1 Introduction

A distributed key–value transaction crosses more boundaries than its small interface suggests. A single logical operation may touch several key ranges, be forwarded to different Raft leaders, enter multiple actor mailboxes, outlive its originating request, race a range split, or be retried after the client has lost the reply. The transaction layer must make all of these mechanisms agree on one outcome while keeping the common case inexpensive.

Kahuna’s answer is a layered protocol. An in-memory coordinator owns the live transaction and its operation history. Key actors stage tentative MVCC values and point locks. For persistent keys, commit converts those transient effects into replicated *prepared intents*. A canonical record, stored on an anchor partition, chooses commit or abort exactly once. Materialization into the ordinary key–value log may happen after the client receives the decision because reads and writes understand unresolved intents. Background recovery, range-reconfiguration barriers, and idempotent replay complete the protocol after failures.

This paper makes four contributions:

1. It presents an implementation-level decomposition of Kahuna’s transaction path, including routing, actor ownership, admission control, MVCC, locking, validation, durable decisions, recovery, and persistence.

2. It derives an anomaly-oriented guarantee matrix rather than assigning one unqualified isolation label to every configuration.
3. It explains the latency and throughput techniques that preserve the protocol’s safety boundaries: an atomic one-partition fast path, partition-local heterogeneous batching, deferred settlement, off-mailbox consensus work, and bounded priority admission.
4. It identifies notable engineering combinations and their limits, including prepared-base fencing, participant-side abort assistance, range-move settlement barriers, and a replica-side stale-base veto whose end-to-end fault campaign is still in progress.

This is a design and implementation paper, not a performance evaluation. Statements about fewer consensus barriers or fewer allocations follow from the code path; they are not substitutes for workload measurements. The analyzed release set is Kahuna 1.5.3, Kommander 1.4.4, Nixie 1.3.0, and CamusDB 0.11.1. Source code and repository documentation are the artifact of record [1, 2, 4, 12].

2 System Model and Scope

2.1 Storage, processes, and failure assumptions

Kahuna partitions the key space. A routing layer maps a key or range to a logical partition and then to the current Raft leader for that partition. Kommander supplies replicated logs, leadership, quorum durability, replay, and snapshot installation using Raft [2, 3]. Nixie supplies bounded-mailbox actors [4]. A per-key actor serializes local state transitions, while an off-mailbox partition scheduler submits replicated writes so that a slow quorum does not block unrelated actor messages.

The transaction protocol assumes crash failures, message loss or delay, retries, leader changes, and range splits or merges. It does not attempt to make progress without an available quorum for the required partition. HLC timestamps provide a causality-respecting, compact time domain [5]; they do not provide Spanner-like bounded clock uncertainty or external consistency [6]. Local monotonic clocks are used for elapsed-time measurements and admission aging, while distributed ordering and durable deadlines are represented by HLC values.

There are two key durability classes:

- **Persistent keys** are represented in Raft and later flushed to the configured materialized backend (memory, SQLite, or RocksDB). Raft/WAL state, not the backend flush, is the immediate durability authority.
- **Ephemeral keys** are memory-resident. Their transactional staging and two-phase coordination do not become crash durable.

A transaction configured for durable decisions rejects any modified ephemeral key. A best-effort transaction may mix the classes, but only its persistent subset is recoverable: a crash between persistent finalization and ephemeral application can expose a partial cross-class outcome. The API therefore makes the durability boundary visible rather than implying atomicity the storage substrate cannot provide.

2.2 Two transaction interfaces

Kahuna exposes two transaction shapes.

Script transactions execute a parsed, server-side sequence. A single statement can take the direct non-transactional path; a multi-statement or explicit transaction uses the coordinator.

Interactive transactions expose begin/read/write/scan/commit/rollback calls over gRPC or the embedded API. The server, not the client, owns the authoritative working set. A handle contains the transaction identifier, coordinator key, and—after the first confirmed persistent mutation—a durable record anchor key. REST exposes script execution and individual key operations, but not the full interactive session lifecycle.

The distinction matters after an ambiguous network outcome. A client-provided list of writes is never accepted as a substitute for the server’s staged state. Reusing the same handle and operation identity allows the server to answer from the live session, a bounded terminal-result cache, or the canonical durable record.

3 Architecture

3.1 Data plane

The normal path has four ownership layers:

```
client request
|
session coordinator and operation registry
|
range/partition router -> current Raft leader
|
per-key actor: MVCC state, lock, staged value
|
partition batch scheduler -> Kommander Raft
```

The coordinator owns transaction-wide facts: timeout, priority lease, read policy, locking policy, participant set, range locks, read observations, operation results, and finalization state. Key actors own key-local facts: committed revisions, the transaction’s staged revision, point locks, write intents, and prepared-intent overlays. This division avoids a shared process-wide transaction object in the hot key path while retaining one authority for commit.

Routing carries a range generation. A write is fenced when it first reaches its key owner and again immediately before a queued replication batch is flushed. This second check is essential: a proposal can wait in a batch long enough for a split, merge, or leader move to invalidate its original route.

3.2 Control plane

Several replicated control structures support the data plane:

- a range map and generation fence determine ownership;
- snapshot holds publish an MVCC retention floor;
- transaction records store canonical decisions;
- prepared-intent records store participant effects;
- completion evidence and transaction metadata survive cache eviction, replay, and range transfer;
- application-durability floors prevent Raft log compaction from outrunning asynchronous backend persistence.

These structures are not merely bookkeeping. They convert local optimizations—caches, asynchronous materialization, and actor-local staging—into recoverable state machines.

3.3 Admission and overload control

Before a script or interactive session begins, it enters a per-node priority gate. The script and session gates are separate so one workload shape cannot consume the other’s entire allowance. Configurable maximum concurrency, queue depth,

and admission wait bound resource use. Reserved slots protect high and critical priorities. Monotonic aging improves queue progress, although the original priority still controls access to reserved capacity.

Admission is a load-shedding mechanism, not a correctness mechanism. It neither preempts an active transaction nor changes lock conflict rules. An `AdmissionRefused` response means no transaction began and the caller may back off. Session lifetime is independently clamped; defaults are a 5 s transaction timeout, a 300 s maximum, a 5 s admission wait, and a 30 s maximum wait. Priority admission is configurable and can be disabled.

The replicated write scheduler has a second overload boundary. It bounds queued items and bytes per partition and globally. Terminal work—decision, materialization, and settlement—has reserved capacity so a flood of new writes cannot indefinitely prevent already-decided transactions from releasing their intents.

4 Live Transaction Execution

4.1 Begin and operation idempotency

Starting a transaction validates the option combination, acquires admission, creates an HLC-derived transaction identifier, and installs a session context. A fixed nonzero read timestamp cannot be combined with tracked latest-read validation because these represent different semantics: a historical as-of view versus a current-version dependency set.

Every interactive operation carries a 128-bit operation identifier and a digest of its request. `BeginOperation` atomically checks the operation table together with the finalization fence. The result is one of three useful cases:

- **new:** the operation may execute;
- **pending:** another copy is executing, so the caller joins its result;
- **completed:** return the saved response without applying the effect again.

Reusing an identifier with a different digest is rejected. A participant-side cache separately handles the case in which a key actor applied an effect but the coordinator did not record completion before the reply was lost. Thus request admission and effect application are both idempotent at their respective ownership layers.

Commit, rollback, and timeout reaping contend for one finalization slot. The winner closes the session to new operations, waits for earlier admitted operations to drain, freezes the transaction snapshot, and performs finalization. Concurrent finalizers observe the same result. This ordering prevents a late write from entering after the participant manifest has been frozen.

4.2 Staging and read-your-writes

A transactional persistent write does not immediately append an ordinary key–value entry to Raft. The key actor checks conditions, acquires an exclusive point lock, and stores an MVCC entry associated with the transaction plus a leased in-memory write intent. Later reads by the same transaction select this staged version, giving read-your-writes behavior without making uncommitted data visible to other transactions.

Client writes take the exclusive point lock in both locking modes. The modes differ primarily in how reads are protected:

- **Pessimistic reads** acquire exclusive point locks. Prefix scans acquire an exclusive prefix lock; range scans acquire a range lock (exclusive by default).
- **Optimistic reads** avoid read locks and record observations for validation. Optimistic locking implies read-set validation even if the explicit validation option is `None`.

The current public point-lock behavior is exclusive rather than shared. Shared per-key read locks are a planned extension, not an implemented guarantee.

4.3 MVCC reads and snapshot holds

Without a fixed timestamp, a read selects the latest visible committed revision plus the transaction’s own staged value. With a fixed read timestamp t_r , point reads and scans select the newest version at or before t_r . This yields a transaction-wide historical as-of view across partitions, provided the required history is retained.

A client that needs a long-lived historical view acquires a replicated, leased snapshot hold. If the live holds have timestamps $h_1, \dots, h_n$, the effective retention floor is

$$F = \min_i h_i.$$

Pruning retains the newest revision at or below F and every newer revision. Actors keep a bounded recent history in memory (16 revisions by default plus the floor boundary); deeper reads fall back asynchronously to the materialized backend rather than blocking an actor mailbox. Holds are reference-counted/leased and reaped after expiration.

This mechanism protects history availability, not serialization of a historical read with a later write. A transaction that reads an old snapshot and then writes is not automatically given optimistic current-version validation. Applications should treat fixed-timestamp transactions as read-only unless they provide their own application-level invariant.

5 Conflict Detection and Validation

Kahuna deliberately separates four questions that are often conflated: who may stage a value, whether an observed version is still current, whether a predicate is protected, and

whether a prepared write is based on the expected predecessor.

5.1 Point locks and dirty writes

An exclusive point lock and write intent allow only one live transaction to stage a write for a key. This prevents dirty writes and makes conflicting writers wait or retry instead of silently overwriting each other's tentative state. The lock is leased and released on commit, abort, or reaping paths. Locks are actor-local and therefore fast, but their liveness depends on renewal and leadership.

Locking alone is insufficient. A transaction can read before taking a write lock, another transaction can commit, and the first can later acquire the lock. Kahuna therefore remembers the base state for read-before-write keys and validates it again during durable preparation.

5.2 Read-set validation

Tracked reads record (key, exists, revision). If the same key is observed with inconsistent base revisions inside one transaction, validation fails immediately. Keys also present in the write set are excluded from ordinary read-set validation because their stronger staged-base fence runs at prepare time.

At commit, the coordinator batches current-state probes across nodes and compares each observed key's existence and revision. A changed or deleted key aborts the transaction. Read-only transactions go through this path as well; skipping validation for an empty write set would admit classic read skew.

Validation is performed after all durable prepares but before the commit decision. Therefore a failed validation leaves durable intents that are safely abortable and recoverable rather than exposing tentative writes.

5.3 Concurrent-writer and range-lock probe

Before read validation, the coordinator makes a batched conflict probe. It asks for:

1. undecided foreign writers on read-only keys; and
2. foreign range or prefix locks covering keys this transaction writes.

Probing active writers first avoids two transactions each waiting for the other's read validation. A missing probe response fails closed. The post-prepare range-lock check is important because a scan can acquire a predicate lock after a writer stages its value but before that writer decides.

Range locks are leader-local, in-memory leases. The coordinator renews them and reacquires them after a participant leader change when the live session can continue. If the coordinator itself is lost, its session and lock list disappear and the leases eventually expire. The durable validation and decision machinery remains the backstop; range locks should not be mistaken for a globally replicated predicate-lock manager.

5.4 Staged-base fencing

For every non-blind persistent write, the prepared intent records whether the base existed and its revision. Kahuna checks this dependency in layers:

1. **Pre-proposal check.** The leader probes the current key state before proposing the prepare. A foreign undecided intent yields retry rather than a guess.
2. **Apply-time fence.** Every replica's prepared-intent state machine compares the validated base with its remembered last transactionally committed head. A competing commit in the probe-to-proposal window makes the prepare stale. The intent is installed deterministically, but the leader withholds a successful prepare acknowledgement, causing the coordinator to abort.
3. **Replica stale-base veto.** If a replica detects a stale base while applying a prepare, it can asynchronously drive a best-effort abort at the canonical anchor. This is intended to prevent a stale leader from acknowledging a prepare that a fresher replica can prove invalid.

Blind writes carry an explicit sentinel instead of a base revision and retain last-writer-wins semantics. The third layer is recent hardening motivated by kill/failover campaigns. Unit and focused regression suites cover it, but the final multi-node fault soak has not yet established closure; Section 11 treats it as promising defense, not settled experimental evidence.

5.5 What anomalies are prevented?

The answer depends on the policy used by the transaction.

Data anomaly	Main defense	Boundary
Dirty read	staged values hidden; decision-aware reads	own writes remain visible to self
Dirty write	exclusive point lock and intent	leased, leader-local live state
Lost update	base revision at prepare/apply	blind writes are last-writer-wins
Non-repeatable point read	pessimistic lock or optimistic validation	no validation in permissive mode
Read skew	frozen read set and commit validation	only observed keys participate
Write skew	read-set validation plus writer probe	dependencies must be read/tracked

Distributed anomaly		Main defense	Boundary
Predicate phantom		prefix/range lock	optimistic scans track returned keys only
Partial durable commit		anchor decision plus durable intents	persistent-only durable mode
Duplicate operation		operation digest, caches	ID, replay caller must reuse identity
Stale route write		generation fence at receipt and flush	retry after re-resolution

Successful pessimistic transactions over correctly locked predicates can protect both points and phantoms. Successful optimistic transactions can provide serializable-like behavior over their observed point dependency set, including read-only transactions and read-before-write keys. However, an optimistic scan records the keys it returned, not the gaps it searched. A new key inserted into that gap can therefore be a phantom. Consequently, it would be incorrect to describe every Kahuna transaction as serializable or snapshot isolated. Kahuna exposes a policy matrix whose guarantees are the conjunction of read timestamp, locking, validation, durability class, and the application’s declared dependencies.

6 The Durable-Intent Commit Protocol

6.1 Frozen commit input

When persistent finalization starts, the coordinator builds an immutable commit input. It contains:

- transaction identifier and epoch;
- coordinator key, anchor key, partition, and route generation;
- one canonical HLC commit timestamp;
- an absolute decision deadline;
- a hash and explicit list of participant keys;
- per-partition prepared intents containing the exact final value or tombstone, target revision, expiration, and validated base state.

The participant manifest prevents recovery from committing a transaction for which only an accidental subset was prepared. Identity is the tuple (transaction, epoch, manifest hash). Exact replays are idempotent; a divergent replay under the same identity is rejected.

The anchor is selected from the first confirmed persistent mutation. Anchoring by key allows normal routing and range

migration to locate the record without a separate global transaction service.

6.2 Replicated state machines

The anchor transaction record has this monotonic state machine:

```
absent --initialize--> undecided --commit-->
committed
\
\--abort----->
aborted
\-----abort tombstone-----> aborted
```

Commit cannot create a record from absence. Abort may create a tombstone, which fences a delayed initialize or commit from resurrecting an expired transaction. An undecided record accepts exactly one terminal decision. An exact terminal replay is a no-op; an identity mismatch fails.

Each participant key has at most one live prepared intent:

```
absent -> pending -> committed -> removed
\-> aborted -> removed
```

The transition is applied deterministically on leaders, followers, WAL replay, and range transfer. An exact duplicate prepare is idempotent. A different effect under the same identity or an intent belonging to another transaction is rejected. Removal is allowed only after resolution has made the effect durable or safely discarded it.

6.3 Standard two-phase path

For a persistent transaction spanning arbitrary partitions, finalization proceeds as follows.

1. **Revalidate staged bases.** Detect obvious lost updates before spending consensus work.
2. **Freeze serialization.** Serialize each exact prepare delta once so retries cannot accidentally change bytes or meaning.
3. **Initialize and prepare.** On the anchor partition, transaction-record initialization and the anchor key’s prepare share one atomic Raft proposal. Other participant partitions prepare concurrently. Completion distinguishes “batch committed” from “this prepare acknowledged,” because a deterministic apply-time fence may commit the proposal while refusing the prepare’s logical success.
4. **Help blocked predecessors.** If a key is occupied by a decided but unsettled intent, consult its canonical record, help materialize or clear it, and retry the idempotent prepare within a bound.
5. **Validate.** After every prepare is quorum durable, run the foreign-writer/range-lock probe and then read-set validation.
6. **Decide.** Propose commit or abort at the anchor. A routed canonical read determines the actual compare-and-set winner; the sender’s local projection is not treated as truth across a leader change.

7. **Resolve.** Materialize committed effects or clear aborted ones, release locks, and settle intent records. With deferred settlement enabled by default, this step may continue after the durable decision is returned.

This ordering produces a useful invariant:

A visible durable commit implies that the canonical record is committed and every persistent participant effect already exists as a quorum-durable prepared intent.

It does not imply that every effect has already been rewritten as an ordinary key-value entry in the backend.

6.4 Adaptive presumed-abort deadline

An undecided 2PC transaction must not block a key forever if the coordinator disappears. Kahuna observes recent finalization latency with a local monotonic timer and derives a window

$$W = \text{clamp}(m \cdot p_{99}, W_{\min}, W_{\max}),$$

where the default multiplier is 4 and the default bounds are 5 s and 60 s. The absolute deadline is $\text{commit HLC} + W$. A fresh HLC attempt timestamp is checked when committing; an attempt beyond the frozen deadline cannot commit.

After the deadline, a participant leader may drive an abort tombstone at the anchor. The tombstone makes presumed abort durable and prevents a delayed coordinator from later choosing commit. The observation window adapts to deployment latency, while the safety comparison lives in the replicated HLC domain rather than depending on synchronized wall clocks.

This is a liveness/safety trade-off. A very short window increases aborts during transient slowness; a long window keeps orphaned intents longer. Neither choice removes the need for an anchor quorum.

6.5 Single-partition fast path

If all persistent participants and the anchor are on one locally led partition, Kahuna can place record initialization, all prepares, and the commit decision in one atomic Raft proposal. Preflight checks foreign intents and validates reads; a late staged-base check narrows the last race before submission. After commitment, the leader installs the committed intent view before replying.

The fast path is disqualified in a multi-process Raft group when the transaction has a read dependency beyond its written keys, or when a write carries a validated base whose safety would otherwise rely on a check performed before a potentially stalled proposal. This conservative rule prevents a queued atomic bundle from applying after a conflicting update that occurred following validation. A single-process Raft group can safely relax some of these constraints because

there is no remote replica divergence or inter-node proposal delay.

Thus the fast path is not merely “one partition means one phase.” Eligibility includes the dependency footprint and execution model. When it cannot prove safety, the coordinator falls back to the standard protocol.

7 Decision-Aware Visibility and Settlement

7.1 Why commit can precede materialization

Synchronous materialization would extend client latency through another replicated write for every participant. Kahuna instead separates the *logical decision* from the *physical representation* of committed data.

Suppose key k has old committed value v_0 and a prepared intent containing v_1 at commit timestamp t_c .

- A snapshot read at $t < t_c$ returns v_0 without consulting the decision.
- A latest read, or snapshot read at $t \geq t_c$, consults the anchor.
- If the record is committed, it overlays v_1 even before materialization.
- If the record is aborted, it returns v_0 .
- If the record is undecided or temporarily unreachable, it waits/retries rather than exposing a guess.

Scans perform the same overlay across intents. A later writer must first resolve a committed predecessor before it derives a new revision or evaluates conditions; it cannot build on stale materialized state.

7.2 Settlement sequence

For commit, a resolver writes the intent’s value as an ordinary transactional key-value log entry, waits for leader application, and only then atomically marks/removes the prepared intent. If any step fails, the intent remains and recovery can retry. For abort, it clears staged state and settles the intent without materializing a value. All operations are identity checked and idempotent.

This ordering handles a crash after decision but before materialization, after materialization but before acknowledgement, and after acknowledgement but before intent removal. In each case the durable record chooses the outcome and at least one durable representation remains available.

Completion evidence lets a participant recognize an already committed operation after its actor state or recent-result cache is gone. The evidence is included in persistence and range transfer so a key cannot lose its transaction identity merely by moving.

7.3 Recovery

Recovery sweeps prepared intents on the current leader of each participant partition:

- Committed record: materialize and settle.
- Aborted record: clear and settle.
- Undecided before deadline: leave it pending.
- Undecided after deadline: attempt canonical abort.
- Missing record after the permitted window: create an abort tombstone.

Every action rechecks the canonical compare-and-set result. A recovery worker that asks for abort but races a valid commit must follow the actual committed decision. The original coordinator is unnecessary once preparation and the anchor record are durable.

8 Reconfiguration, Leadership, and Persistence

8.1 Leader changes

Actor caches and live locks are per node; they are not magically coherent across replicas. Correctness after promotion comes from replaying deterministic transaction-record and prepared-intent state, then consulting those structures rather than trusting a cache. Requests are routed to current leaders, and canonical record reads may be re-routed when leadership changes.

The difficult case is a promoted replica with an orphaned prepared intent and an older per-key committed-head memory than another replica observed. The apply-time base fence only helps if the leader evaluating the logical acknowledgment has the necessary history. The replica stale-base veto extends detection to any applying replica and asks it to force an anchor abort. This defense directly targets the gap, but its final partition/kill soak remains an open validation item.

Live range and point locks are not reconstructed as durable lock ownership. They expire or are reacquired by a surviving coordinator. Safety after a coordinator loss instead rests on durable prepared intents, base validation, the canonical decision, and recovery.

8.2 Range splits and merges

An intent cannot be moved like an ordinary value unless the destination also retains everything needed to resolve it. Kahuna therefore establishes a settle-before-cutover barrier:

1. scan the moving interval for prepared intents;
2. resolve every decided intent;
3. refuse transfer while an undecided intent remains or the scan cannot be trusted;
4. transfer values, completion evidence, and remaining metadata;
5. change the routing generation.

Materialization itself re-resolves the key's current partition rather than assuming the partition captured at prepare time. These two measures address opposite races: the barrier prevents a move from dropping an overlay, and re-resolution prevents a late resolver from writing to the old owner.

Every queued write also carries its range generation and is rechecked at batch flush. A stale operation is released for routing retry without contaminating valid siblings in the same batch.

8.3 Raft versus materialized storage

Kommander's replicated WAL establishes the immediate durable order. A background writer later persists committed key revisions and transaction metadata to the selected backend. Reads can use resident replicated state before that flush. An application-durability floor prevents Raft compaction from discarding log entries that the backend has not yet made independently recoverable.

This arrangement separates consensus latency from backend write amplification, but it also creates an invariant: snapshot/compaction policy must advance only behind materialization. Treating the backend as current merely because the client saw commit would be incorrect.

9 Performance-Oriented Design

9.1 Heterogeneous partition batching

Kahuna's partition scheduler accepts typed submissions—ordinary key writes, transaction records, and prepared intents—and coalesces independent operations headed to the same partition into one Kommander `ReplicateEntries` call. A default 1 ms oldest-item linger allows a small batching window; batches are capped at 512 items and 4 MiB. Oversized valid items run alone. Only one batch is in flight per partition, while different partitions progress concurrently.

Unlike transaction-level batching, heterogeneous coalescing does not require unrelated clients to share a transaction. Each item retains its own result and range fence, while the shared proposal amortizes serialization, network, quorum, and WAL costs. Client cancellation stops waiting for that item but does not cancel a consensus batch shared with other clients. Accepted items complete exactly once at the scheduler boundary.

Kommander can additionally group WAL work across partitions. These two layers exploit different locality: Kahuna groups logical entries per Raft partition; the WAL layer may group physical persistence across partitions.

9.2 Consensus-barrier reduction

On the standard path, the anchor initialization and anchor prepare share a proposal, and non-anchor prepares run concurrently. The decision requires a subsequent anchor proposal. On an eligible single-partition transaction, initializa-

tion, preparation, and decision share one proposal. Deferred settlement removes materialization from the client-visible critical path in either case.

Barrier counting is more informative here than claiming a universal latency number. Network topology, quorum placement, batch density, backend choice, contention, and read validation all affect measured latency. The structural result is that the common local case needs one consensus decision barrier, while a multi-partition commit needs prepare durability plus a terminal decision barrier.

9.3 Actor and allocation discipline

Per-key actors make key-local state mutation sequential and keep critical sections explicit. Consensus I/O is submitted off mailbox, so unrelated reads or lock-release messages need not wait for a network round trip. Participant work is grouped by partition, validation uses batched probes rather than one remote request per key, and prepare deltas are serialized once before retry loops.

These choices reduce obvious N-by-participant network patterns and repeated serialization. Bounded mailboxes, queue byte caps, transaction timeouts, participant-effect leases, and terminal reservations constrain retained state under overload. They also make overload visible as backpressure or retry instead of unbounded allocation.

9.4 Costs of the design

The optimizations do not make transactions free. A persistent transaction creates an anchor record and one prepared intent per written key, performs validation probes, and later materializes and removes each intent. Deferred settlement moves work out of foreground latency but can lengthen the interval during which readers consult transaction records and writers help predecessors. Heterogeneous batching trades up to the linger interval for density. Pessimistic scans can restrict concurrency over large ranges. Snapshot holds increase revision retention and backend space.

The system exposes metrics for queueing, batch fill, settlement failure, recovery, WAL phases, and related paths so these trade-offs can be measured rather than inferred from throughput alone.

10 API Semantics Under Ambiguity

Distributed clients must interpret outcomes conservatively:

- **Committed:** the canonical persistent decision is commit; participant effects were prepared before that decision. Materialization may still be running.
- **Aborted:** a canonical terminal abort exists, including conflict, explicit rollback, validation failure, or presumed abort. The caller must start a new transaction for new work.
- **MustRetry:** no safe terminal answer was established at this boundary. The caller should retry with the same trans-

action handle and, for an operation, the same operation identifier.

- **AdmissionRefused:** no transaction started; backoff or lower load.

A timeout or lost connection is not proof of abort. Starting an unrelated replacement transaction can duplicate application effects if the original later proves committed. Durable handles, operation digests, terminal-result retention, and canonical anchor lookup are intended to make same-identity retry the natural safe behavior.

Direct non-transactional writes have a different retry contract. If the transport reports an indeterminate outcome and the operation lacks an application idempotency key, blindly replaying it can repeat a condition or mutation. The stronger operation registry belongs to interactive transactions.

11 Relationship to Prior Systems

Kahuna's foundation is conventional in the best sense: Raft replicates partition state [3], HLCs represent causality-compatible timestamps [5], and two-phase commit separates participant preparation from a terminal decision [7]. Its interest lies in how those tools are composed around a key-actor data path.

Percolator implements distributed transactions over a key-value substrate using timestamped locks and a distinguished primary [8]. Kahuna's anchor record plays a related decision-authority role, but its record is an explicit replicated state machine, its participant effects are typed Raft entries, and its live coordinator supports both locking and validation policies.

CockroachDB also uses transaction records, write intents, MVCC, and asynchronous intent resolution [9]. Kahuna shares the principle that a committed logical value can be read through an unresolved intent. Its distinctive implementation emphasis is the integration with per-key actors, heterogeneous partition proposal batching, explicit range-generation fences, and configurable server-owned interactive sessions.

FoundationDB uses optimistic conflict ranges and presents strict serializability [10]. Kahuna's optimistic point validation is conceptually similar to checking declared dependencies, but Kahuna does not presently turn every optimistic scan into a conflict range. Its guarantee is therefore intentionally more conditional.

Spanner combines 2PC, Paxos replication, MVCC, and True-Time to supply externally consistent transactions [6]. Kahuna uses HLC without a bounded-uncertainty commit-wait protocol and makes no external-consistency claim.

Calvin avoids distributed commit coordination in its execution phase by deterministically ordering transactions in advance [11]. Kahuna instead supports dynamic, interactive

transactions whose read and participant sets emerge during execution; it pays prepare/decision coordination to preserve that flexibility.

No claim is made that any individual mechanism is historically novel. The notable features are engineering combinations:

- atomic anchor initialization plus anchor preparation, with an eligibility-checked one-proposal commit fast path;
- exact-value prepared intents that double as the read overlay during deferred settlement;
- three-layer lost-update defense culminating in participant-replica abort assistance;
- adaptive HLC decision deadlines that make presumed abort a replicated state transition;
- range-move refusal until decision overlays are safe to transfer;
- heterogeneous batching across unrelated ordinary and transactional entries while preserving per-item fencing and completion;
- a server-owned idempotency registry joined atomically with finalization admission.

Their value is not novelty in isolation, but the reduction of gaps between the transaction protocol and the operational realities of actors, retries, reconfiguration, and asynchronous persistence.

12 Limitations and Threats to Validity

First, this analysis is tied to one source snapshot and its documented tests. It is not a formal proof. The code contains deterministic state machines and focused fault tests, but an implementation can still violate a derived invariant through an unmodeled path.

Second, the strongest isolation statement is configuration dependent. Optimistic point reads are validated, but optimistic scans validate returned keys rather than gaps; phantoms remain possible. Pessimistic predicate locks are leased, in-memory, and leader-local. Fixed historical snapshots protect time-consistent reads and retention, not automatic serialization of later writes. Shared point-read locks are not yet implemented.

Third, crash atomicity covers the persistent participant set in durable mode. Ephemeral participants are intrinsically unrecoverable, and mixed best-effort transactions can become partially visible after a crash.

Fourth, 2PC availability remains bounded by consensus and decision recovery. A partition that cannot reach quorum cannot safely decide. Presumed abort limits indefinite orphan blocking after a deadline; it does not create availability during quorum loss.

Fifth, the most recent per-key failover hardening needs further empirical validation. A partition/kill campaign found

executions in which a promoted node combined an orphaned prepared intent with stale per-key committed-head state, allowing acknowledged results to fork after failover. Kahuna 1.5.3 adds a replica-side stale-base veto and focused tests are green, but the repository’s acceptance gate still calls for an extended multi-node fault soak. Until that gate passes, the proper claim is “designed and locally validated to close the stale-leader gap,” not “demonstrated invariant under the full campaign.”

Finally, this paper reports no fresh benchmark. The one-phase path, batching, and deferred settlement remove or amortize identifiable work; actual throughput, tail latency, recovery time, and storage amplification require reproducible experiments across contention levels, participant counts, backends, and fault schedules.

13 Conclusion

Kahuna’s persistent transaction system is organized around a compact safety idea: prepare every exact participant effect durably, choose one canonical outcome, and make every read, retry, recovery worker, and range move honor that outcome until ordinary storage catches up. The surrounding subsystems exist to preserve that idea at their boundaries. Actor ownership prevents local races; operation identities prevent duplicate effects; locks and validation protect declared dependencies; base fences close read-to-prepare races; Raft makes records and intents replayable; decision-aware reads permit deferred materialization; deadlines and tombstones resolve abandoned work; and range barriers keep reconfiguration from dropping unresolved state.

The design brings a durable, recoverable transaction core to a high-performance actor-based key-value layer without forcing every workload through the same concurrency policy. That flexibility is also the reason its guarantees must be stated precisely. Persistent-only durable transactions, complete dependency tracking, and predicate locking where phantoms matter can support strong executions. Permissive reads, optimistic scans, historical writes, or ephemeral participants deliberately weaken the result. The clearest description is therefore not a single isolation label, but a protocol core plus an explicit guarantee envelope.

14 References

- [1] Kahuna Project. *Kahuna source tree*, version 1.5.3, 2026. <https://github.com/kahunakv/kahuna>
- [2] Kahuna Project. *Kommander source tree*, version 1.4.4, 2026. <https://github.com/kahunakv/kommander>
- [3] D. Ongaro and J. Ousterhout. “In Search of an Understandable Consensus Algorithm (Extended Version).” *USENIX ATC*, 2014. <https://web.stanford.edu/~ouster/cgi-bin/papers/raft-extended.pdf>

- [4] Kahuna Project. *Nixie source tree*, version 1.3.0, 2026. <https://github.com/kahunakv/nixie>
- [5] S. S. Kulkarni, M. Demirbas, D. Madappa, B. Avva, and M. Leone. “Logical Physical Clocks and Consistent Snapshots in Globally Distributed Databases.” *OPODIS*, 2014. <https://cse.buffalo.edu/~demirbas/publications/hlc.pdf>
- [6] J. C. Corbett et al. “Spanner: Google’s Globally-Distributed Database.” *OSDI*, 2012. <https://research.google/pubs/spanner-googles-globally-distributed-database-2/>
- [7] J. Gray and L. Lamport. “Consensus on Transaction Commit.” *ACM Transactions on Database Systems*, 31(1), 2006. <https://arxiv.org/abs/cs/0408036>
- [8] D. Peng and F. Dabek. “Large-scale Incremental Processing Using Distributed Transactions and Notifications.” *OSDI*, 2010. <https://research.google/pubs/large-scale-incremental-processing-using-distributed-transactions-and-notifications/>
- [9] Cockroach Labs. “CockroachDB Architecture and Design.” <https://github.com/cockroachdb/cockroach/blob/master/docs/design.md>
- [10] J. Zhou et al. “FoundationDB: A Distributed Unbundled Transactional Key Value Store.” *SIGMOD*, 2021. <https://www.foundationdb.org/files/fdb-paper.pdf>
- [11] A. Thomson et al. “Calvin: Fast Distributed Transactions for Partitioned Database Systems.” *SIGMOD*, 2012. <https://dsf.berkeley.edu/cs286/papers/calvin-sigmod2012.pdf>
- [12] Kahuna Project. *CamusDB source tree*, version 0.11.1, 2026. <https://github.com/kahunakv/camusdb>

15 Appendix: Implementation Map

The following source areas are the shortest path for reproducing the paper’s implementation trace:

- `Kahuna.Core/KeyValues/Transactions/TransactionCoordinator.cs`: session life-cycle, finalization, validation ordering, and retry outcome mapping.
- `Kahuna.Core/KeyValues/Transactions/Data/TransactionContext.cs`: authoritative working set, read observations, range locks, and operation registry.
- `Kahuna.Core/KeyValues/Transactions/DurableTransactionFinalizer.cs`: standard prepare/decide/resolve path and one-partition fast path.
- `Kahuna.Core/KeyValues/Transactions/DurableFinalizeInputBuilder.cs`: frozen participant manifest, commit timestamp, deadline, and base-revision encoding.
- `Kahuna.Core/KeyValues/Transactions/TransactionRecordStore.cs`: canonical decision state machine.

- `Kahuna.Core/KeyValues/Transactions/PreparedIntentStore.cs`: participant state machine, uniqueness, base fence, and replay.
- `Kahuna.Core/KeyValues/Transactions/DurableTransactionRecovery.cs`: decision lookup, presumed abort, materialization, and settlement.
- `Kahuna.Core/KeyValues/KeyValueActor.cs`: key-local MVCC, locks, transactional staging, and intent overlays.
- `Kahuna.Core/KeyValues/Writes/`: bounded heterogeneous partition batching and Raft submission.
- `Kahuna.Core/KeyValues/SnapshotHoldService.cs` and `Ranges/SnapshotFloorStore.cs`: replicated snapshot holds and the MVCC retention floor.
- `Kahuna.Core/KeyValues/LocalLockOperations.cs`, `RoutedLockOperations.cs`, and `Handlers/RangeLockChecks.cs`: point, prefix, and range lock ownership and lease behavior.
- `Kahuna.Core/KeyValues/Ranges/DataPartitionRouter.cs` and `RangeStateTransferService.cs`: routing generation and split/merge fencing.